Journal Trend Analysis - Source Code Structure & Technical Report

Project: journal_trend_analysis
Framework: Flutter (Dart SDK ^3.12.0)
Author: PRM393 Lab
Last updated: June 18, 2026

---

1. Application Overview

Journal Trend Analyzer is a Flutter cross-platform application for searching, ranking, and analyzing scientific publications from the OpenAlex REST API. The project follows a Clean Architecture style: UI code reads Riverpod providers, providers call use cases and repositories, and only the data layer communicates with OpenAlex through Dio.

The current application focuses on four mounted user workflows:

- Search publications by free-text keyword.
- Search publications by OpenAlex topic hierarchy filters: domain, field, subfield, and topic.
- Analyze publication trends, KPIs, top journals, and top authors from the active result set.
- Visualize geographic/institution research distribution and author collaboration networks.

Important project change: the previous report described Dashboard and Top Papers as primary navigation tabs. The current router mounts Search, Trends, Heatmap, and Network. Dashboard and Top Papers screens still exist in the codebase, but they are not currently connected to the bottom navigation.

---

2. Current Source Structure

```
journal_trend_analysis/
|-- lib/
|   |-- main.dart
|   |-- core/
|   |   |-- constants/
|   |   |   `-- api_constants.dart
|   |   |-- router/
|   |   |   `-- app_router.dart
|   |   |-- theme/
|   |   |   |-- app_colors.dart
|   |   |   |-- app_dimensions.dart
|   |   |   |-- app_text_styles.dart
|   |   |   `-- app_theme.dart
|   |   `-- utils/
|   |       `-- formatter.dart
|   |-- data/
|   |   |-- api/
|   |   |   `-- openalex_api_client.dart
|   |   |-- datasources/
|   |   |   |-- heatmap_remote_datasource.dart
```

```
|   |   |   |-- publication_remote_datasource.dart
|   |   |   `-- topic_remote_datasource.dart
|   |   |-- models/
|   |   |   |-- author_model.dart
|   |   |   |-- journal_model.dart
|   |   |   `-- publication_model.dart
|   |   `-- repositories/
|   |       `-- publication_repository_impl.dart
|   |-- domain/
|   |   |-- entities/
|   |   |   |-- author.dart
|   |   |   |-- heatmap_data.dart
|   |   |   |-- journal.dart
|   |   |   |-- paginated_result.dart
|   |   |   |-- publication.dart
|   |   |   `-- topic_hierarchy.dart
|   |   |-- repositories/
|   |   |   `-- publication_repository.dart
|   |   `-- usecases/
|   |       |-- get_dashboard_summary.dart
|   |       |-- get_top_authors.dart
|   |       |-- get_top_journals.dart
|   |       |-- get_trend_data.dart
|   |       `-- search_publications.dart
|   `-- presentation/
|       |-- providers/
|       |   |-- heatmap_providers.dart
|       |   `-- providers.dart
|       |-- screens/
|       |   |-- author_network_screen.dart
|       |   |-- dashboard_screen.dart
|       |   |-- heatmap_screen.dart
|       |   |-- publication_detail_screen.dart
|       |   |-- search_screen.dart
|       |   |-- top_papers_screen.dart
|       |   `-- trend_analysis_screen.dart
|       `-- widgets/
|           |-- author_chip.dart
|           |-- empty_state.dart
|           |-- error_state.dart
|           |-- metric_card.dart
|           |-- publication_card.dart
|           |-- ranked_list_tile.dart
|           |-- shimmer_loader.dart
|           |-- topic_cascade_dialog.dart
|           `-- trend_chart.dart
|-- assets/
|   `-- icon/app_icon.png
|-- android/
|-- ios/
|-- linux/
|-- macos/
|-- web/
|-- windows/
|-- pubspec.yaml
`-- test/
    `-- widget_test.dart
```

---

3. Architecture

3.1 Core Layer

The core layer contains shared configuration and app-wide infrastructure:

- `ApiConstants` stores the OpenAlex base URL, timeouts, and default page size.
- `app_router.dart` defines navigation with `GoRouter` and `ShellRoute`.
- Theme files centralize Material 3 colors, spacing, typography, and `ThemeData`.
- `formatter.dart` handles formatting citation counts, numbers, DOI display, and abstract reconstruction.

3.2 Domain Layer

The domain layer is framework-light business logic:

- `Publication`, `Author`, `Journal`, `PaginatedResult`, `TopicHierarchyItem`, and heatmap entities describe app data.
- `PublicationRepository` is the abstract contract used by use cases and providers.
- Use cases perform pure app calculations:
  - `SearchPublications` validates query/page parameters and delegates search.
  - `GetTrendData` groups publications by publication year and totals citations.
  - `GetDashboardSummary` calculates KPIs such as average citations, most active year, top author, and top journal.
  - `GetTopAuthors` aggregates author publication/citation counts.
  - `GetTopJournals` aggregates journal publication/citation counts.

3.3 Data Layer

The data layer owns network access and JSON parsing:

- `OpenAlexApiClient` wraps Dio with base URL, 15-second connect/receive timeouts, JSON headers, and OpenAlex polite-pool `User-Agent`.
- `PublicationRemoteDataSource` calls `/works` for publication search, topic-filter search, and top papers.
- `TopicRemoteDataSource` calls `/domains`, `/fields`, `/subfields`, `/topics`, and autocomplete endpoints for topic discovery.
- `HeatmapRemoteDataSource` calls `/works` with `group_by` to aggregate countries and institutions.
- `PublicationRepositoryImpl` converts data models into domain entities and wraps paginated results.

3.4 Presentation Layer

The presentation layer contains Riverpod providers, screens, and reusable UI widgets:

- `providers.dart` wires dependency injection, search state, pagination, topic filters, derived trend data, dashboard summary, rankings, and sort option.
- `heatmap_providers.dart` wires heatmap data source and country/institution view state.
- Screens consume providers with `ConsumerWidget` or `ConsumerStatefulWidget`.
- Shared widgets render loading, error, empty, ranking, metric, publication card, and trend chart states.

---

4. Navigation and Screen Flow

Current router configuration:

```
ShellRoute with persistent NavigationBar
|-- /search    -> SearchScreen
|-- /trends    -> TrendAnalysisScreen
|-- /heatmap   -> HeatmapScreen
`-- /network   -> AuthorNetworkScreen

Standalone push route:
`-- /publication/:id -> PublicationDetailScreen
```

Navigation details:

- The app starts at `/search`.
- Search, Trends, Heatmap, and Network share one bottom `NavigationBar`.
- Publication detail is opened with `context.push('/publication/:id', extra: publication)`.
- Publication detail depends on the full `Publication` object passed through `state.extra`.

Screens present in code but not mounted by the current router:

- `dashboard_screen.dart`
- `top_papers_screen.dart`

These files can be reconnected later if the product wants separate Dashboard and Top Papers tabs again. For now, dashboard KPIs and top author/journal ranking are integrated into `TrendAnalysisScreen`.

---

5. Feature Analysis

5.1 Search Screen

`SearchScreen` is the main data entry point. It supports:

- Free-text search with debounce-driven autocomplete display.
- Topic autocomplete across domains, fields, subfields, and topics.
- Hierarchical drawer for browsing OpenAlex topic levels.
- End drawer filters and result sorting.
- Infinite pagination through a "Read more" button.
- Accumulated result list stored in widget state.
- Search chips for authors, journals, and research topics.
- Tap-to-open publication details.

Search state is coordinated by:

- `searchQueryProvider`
- `selectedTopicFilterProvider`
- `searchPageProvider`
- `searchPerPageProvider`
- `paginatedPublicationsProvider`
- `publicationsProvider`
- `sortedPublicationsProvider`

Free-text search uses `/works?search=...&sort=relevance_score:desc`. Topic hierarchy filters use `/works?filter=<filterKey>:<id>&sort=cited_by_count:desc`.

5.2 Trend Analysis Screen

`TrendAnalysisScreen` derives analysis from the active search result set:

- KPI row: total papers, average citations, most active year.
- Line chart for publications per year.
- Year-range filter dialog.
- Top Journals and Top Authors sections.
- Ranking mode toggle: papers or citations.
- "Show more" controls for longer rankings.
- Journal/author taps route back to Search with a new query.

Supporting providers:

- `trendDataProvider`
- `dashboardSummaryProvider`
- `topAuthorsProvider`
- `topJournalsProvider`
- `trendYearRangeProvider`
- `trendRankModeProvider`
- `filteredTrendDataProvider`

5.3 Heatmap Screen

`HeatmapScreen` visualizes research distribution for the active search context:

- Country mode and Institution mode through `SegmentedButton`.
- Country mode supports world map and grid display.
- Country data is ranked by OpenAlex grouped work count.
- Institution mode lists top institutions with proportional bars.
- Empty, loading, and error states are handled with reusable widgets.

OpenAlex aggregation uses:

- `group_by=authorships.countries`
- `group_by=authorships.institutions.lineage`

5.4 Author Network Screen

`AuthorNetworkScreen` builds a collaboration graph from the loaded publications:

- Aggregates author nodes by ID or display name.
- Builds co-author edges for every author pair in a publication.
- Keeps the top 30 authors to reduce graph overcrowding.
- Supports scale by paper count or citation count.
- Uses a custom force-layout simulation and `CustomPainter`.
- Supports pan/zoom with `InteractiveViewer`.
- Supports dragging author nodes.
- Tapping an edge opens a bottom sheet listing shared publications.

This feature is computed locally from the current in-memory result set and does not call a separate API endpoint.

5.5 Publication Detail Screen

The detail screen receives a `Publication` entity from route `extra` and displays publication

metadata:

- Title, authors, journal, year, citation count.
- DOI and external link behavior through `url_launcher`.
- Abstract reconstructed from OpenAlex `abstract_inverted_index`.
- Concepts/topics associated with the work.

---

6. Data and State Flow

```
User action
  -> SearchScreen updates searchQueryProvider or selectedTopicFilterProvider
  -> paginatedPublicationsProvider calls SearchPublications or repository filter
search
  -> PublicationRepositoryImpl calls PublicationRemoteDataSource
  -> OpenAlexApiClient performs Dio request
  -> PublicationModel.fromJson parses OpenAlex JSON
  -> model.toEntity() returns Publication domain entities
  -> providers derive trend, dashboard, author, journal, heatmap, and network
views
  -> screens rebuild reactively
```

Key implementation details:

- `PaginatedResult<T>` tracks `items`, `totalCount`, `page`, `perPage`, `totalPages`, `hasNextPage`, and `hasPreviousPage`.
- `Publication` includes `countsByYear`, `abstractInvertedIndex`, concepts, authors, journal name, DOI, and citation count.
- Search pagination defaults to 50 items per page through `searchPerPageProvider`.
- Derived providers are synchronous once publication data is loaded.
- Heatmap providers are asynchronous because they use OpenAlex `group_by` endpoints separately from publication search.

---

7. OpenAlex API Integration

Use case	**Endpoint**	**Main parameters**
Free-text publication search	`GET /works`	`search`, `page`, `per_page`, `sort=relevance_score:desc`
Topic-filter publication search	`GET /works`	`filter=<primary_topic...>:<id>`, `page`, `per_page`, `sort=cited_by_count:desc`
Top papers	`GET /works`	`per_page=50`,

Use case	Endpoint	Main parameters
		`sort=cited_by_count:desc`, optional `filter=concepts.display_name:<topic>`
Topic autocomplete	`GET /domains`, `/fields`, `/autocomplete/subfields`, `/autocomplete/topics`	`search` or `q`, small result limits
Topic hierarchy drawer	`GET /domains`, `/fields`, `/subfields`, `/topics`	`filter` by parent level, `select=id,display_name,works_count`
Country heatmap	`GET /works`	`group_by=authorships.countries`, optional `search` or topic `filter`
Institution heatmap	`GET /works`	`group_by=authorships.institutions.lineage`, optional `search` or topic `filter`

Response parsing highlights:

- `results[]` becomes `PublicationModel`.
- `primary_location.source.display_name` becomes `journalName`.
- `authorships[].author` becomes `AuthorModel`.
- `concepts[].display_name` becomes publication concepts.
- `counts_by_year[]` becomes `YearlyCitation`.
- `abstract_inverted_index` is preserved and reconstructed later for display.
- `meta.count` becomes `PaginatedResult.totalCount`.

---

8. Main Libraries and Roles

Library	Role in project
`flutter_riverpod`	Dependency injection and reactive state
`go_router`	Declarative routing, shell navigation, detail route
`dio`	HTTP client for OpenAlex
`fl_chart`	Trend line chart
`shimmer`	Skeleton loading UI
`google_fonts`	Typography

Library	Role in project
`url_launcher`	Opening DOI/external links
`countries_world_map`	Interactive world map heatmap
`equatable`	Value equality for domain entities
`freezed_annotation`, `json_annotation`	Available for generated immutable/JSON models, though current shown models are manually parsed
`flutter_launcher_icons`	Platform app icon generation

---

9. Design Patterns Used

Pattern	Current usage
Clean Architecture	Separates core, data, domain, and presentation concerns
Repository Pattern	`PublicationRepository` and `PublicationRepositoryImpl`
Dependency Injection	Riverpod providers construct API client, data sources, repositories, and use cases
Observer/Reactive State	Screens rebuild from provider changes
Factory Method	`PublicationModel.fromJson`, author/journal model parsing
Strategy-like enum	`PaperSortOption`, `TrendRankMode`, `HeatmapViewMode`, country display mode
Composition	Screens are assembled from reusable widgets and providers
Local aggregation	Trend, dashboard, rankings, and author network are computed from loaded publications

---

10. Error Handling and UX States

The app consistently handles asynchronous states:

State	Widget or behavior
Loading	`ShimmerLoader` or `CircularProgressIndicator`
Error	`ErrorState`, retry buttons, and snackbars for pagination errors
Empty	`EmptyState` with contextual message

State	Widget or behavior
Data	Screen-specific content

Examples:

- Search shows shimmer for initial loading and preserves already loaded results while loading more.
- Trends shows an empty state until a topic/search result exists.
- Heatmap shows an empty state when no search query is active or no geographic data exists.
- Network shows an empty state when no publications or author data are loaded.

---

11. Functional and Structural Changes Found

Compared with the previous report, the current codebase has these important changes:

- Added topic hierarchy support through `TopicHierarchyItem`, `TopicRemoteDataSource`, autocomplete providers, and a hierarchy drawer.
- Added paginated search through `PaginatedResult`, `searchPageProvider`, and "Read more" accumulation in `SearchScreen`.
- Added heatmap functionality through `HeatmapRemoteDataSource`, `heatmap_providers.dart`, and `HeatmapScreen`.
- Added author collaboration network visualization through `AuthorNetworkScreen`.
- Added `countsByYear` parsing to `PublicationModel` and `Publication`.
- Changed active navigation tabs to Search, Trends, Heatmap, and Network.
- Moved dashboard-style KPIs and top author/journal rankings into the Trends page.
- Kept Dashboard and Top Papers screens in the repository but removed them from active router navigation.
- Added `countries_world_map` dependency for world map rendering.
- Updated publication search sorting: free text uses relevance sorting, topic filters use citation sorting.

---

12. Technology Summary

Category	Technology
Framework	Flutter 3.x, Dart SDK ^3.12.0
State management	Riverpod 2.x
Navigation	GoRouter 13.x

Category	Technology
HTTP client	Dio 5.x
API	OpenAlex REST API
Charts	FL Chart 0.68.x
Map visualization	countries_world_map 1.3.x
Loading UX	Shimmer 3.x
External links	url_launcher 6.x
Typography	Google Fonts 6.x
Value equality	Equatable 2.x
App icon tooling	flutter_launcher_icons 0.14.x
Architecture	Clean Architecture with repository and use-case layers
Supported platforms	Android, iOS, Web, Windows, Linux, macOS

---

13. Maintenance Notes

- If Dashboard and Top Papers are still required by the lab specification, reconnect them in `app_router.dart` and update the `NavigationBar` destinations.
- The project imports generated-code packages, but the current inspected models are manually implemented. Remove unused generation dependencies or introduce generated models consistently.
- `PublicationDetailScreen` assumes `state.extra` is a `Publication`; deep linking directly to `/publication/:id` without `extra` can fail.
- Heatmap institution country codes are currently empty because OpenAlex institution lineage IDs do not directly include country information.
- Several source comments still contain mojibake characters. Cleaning comments would improve readability without changing behavior.

End of Report